

Le jeu d'Hex

Cahier des charges

7 janvier 2025
Version 0.1

Ce projet a pour objectif de développer un logiciel permettant de jouer à Hex à deux joueurs. Le programme doit permettre aux joueurs de s'affronter sur un plateau de jeu en respectant les règles.

Mots-clefs: Deux joueurs, déterministe, information parfaite.

1 Généralités

F1. Langage de programmation

Le langage de programmation de ce projet doit être choisi parmi les langages suivants :

- **Langage C** : C11 (ISO/IEC 9899 :2011).
- **Langage Java** : Java SE 17+.
- **Langage Python** : Python 3.7+.

F2. Style de codage

Le *coding style* du programme doit respecter :

- **Langage C** : le *coding style* de GNU.
- **Langage Java** : le *coding style* de Google.
- **Langage Python** : le *coding style* du PEP8.

F3. Langue par défaut dans le code

La documentation, le nom des variables, des fonctions et des fichiers devront être en anglais.

F4. Système cible

Le programme devra fonctionner sur des systèmes d'exploitation GNU/Linux.

F5. Documentation

Le programme et le code source devra être documenté : manuel utilisateur, option d'aide en ligne de commande et commentaires dans le code source.

F6. Tests

Le programme devra être testé et validé par suffisamment de tests pour garantir son bon fonctionnement (couverture d'au moins 80%). Les tests doivent être automatisés et facile à lancer.

F7. Bugs

Aucun bug menant à un crash ne doit être présent dans le programme. Les spécifications doivent être respectées et les erreurs doivent générer des messages d'erreur.

F8. Performances

Les performances du programme doivent être mesurées et documentées pour différentes entrées du programme ainsi que pour ses cas limites.

2 Bibliothèques et outils

F9. Build-system

Le *'build-system'* devra être basé sur :

- **Langage C** : CMake.
- **Langage Java** : Maven.
- **Langage Python** : pyproject.toml + setuptools.

F10. Frameworks de tests

Le *framework* de test sera basé sur :

- **Langage C** : CTest + googletest + gcovr.
- **Langage Java** : JUnit + Jacoco.
- **Langage Python** : pytest + coverage.

F11. Gestion des options

Les options en ligne de commande seront gérées par :

- **Langage C** : le module getopt_long.
- **Langage Java** : la bibliothèque commons-cli.
- **Langage Python** : le module argparse.

F12. Bibliothèque graphique

La bibliothèque graphique utilisé sera basé sur :

- **Langage C** : LibGtk 4.x.
- **Langage Java** : JavaFx.
- **Langage Python** : PyGObject.

F13. Internationalisation

Le *framework* d'internationalisation sera basé sur :

- **Langage C** : la bibliothèque gettext.
- **Langage Java** : ResourceBundle et Locale.
- **Langage Python** : le module Python gettext.

La langue par défaut, sans locale précisée, sera l'anglais et, au moins, le français sera ajouté.

3 Options en ligne de commande

F14. Nom de l'exécutable principal

L'exécutable principal doit s'appeler 'hex' et permettre de lancer tout le programme.

F15. Usage général

Le programme sera lancé comme ceci :

```
#> hex [OPTIONS] [FILENAME]
```

Si une option est incorrecte, le programme devra afficher un message d'erreur et l'aide du programme.

Si un nom de fichier est donné, le programme devra charger une partie depuis ce fichier. Sinon, le programme devra démarrer une nouvelle partie.

Les messages à l'utilisateur seront écrits sur la sortie standard (stdout) et les messages d'erreurs devront être écrits sur la sortie d'erreur standard (stderr). De plus, le code de retour du programme devra être EXIT_SUCCESS en cas de succès et EXIT_FAILURE en cas d'erreur.

F16. Aide en ligne de commande

L'option '-h', '--help' affiche l'aide puis quitte.

F17. Version

L'option '-V', '--version' affiche la version puis quitte.

F18. Mode verbose

L'option '-v', '--verbose' permettra d'afficher des informations supplémentaires pendant l'exécution.

F19. Mode debug

L'option '-d', '--debug' permettra d'afficher des informations de débogage pendant l'exécution.

F20. Mode blitz

L'option '-b', '--blitz' permettra de jouer en mode blitz. Dans ce mode, un temps limite sera donné à chaque joueur pour jouer un coup. Si le temps d'un des joueur est écoulé, il perd la partie.

F21. Durée du blitz

L'option '-t TIME', '--time TIME' permettra de spécifier le temps limite en minutes pour chaque joueur en mode blitz. Par défaut, le temps limite sera de 30 minutes. Si cette option est donnée sans l'option '-b', '--blitz', elle sera ignorée mais un message d'avertissement sera affiché.

F22. Changement de couleur

L'option '-s', '--swap' permettra au second joueur lors du premier tour de choisir s'il échange sa couleur avec son adversaire ou pas. Par défaut, cette option est désactivée.

F23. Taille du plateau

L'option '-z SIZE', '--size SIZE' permettra de spécifier la taille du plateau de jeu (SIZE ne pourra pas être en dessous de 1, ni au-dessus de 20). Par défaut, la taille du plateau sera de 11 cases.

F24. Mode contest

L'option '-c FILENAME', '--contest FILENAME' permet de lancer le programme en mode 'contest' en chargeant une partie depuis un fichier 'FILENAME'. Le programme devra alors afficher le joueur et le coup à jouer au format décrit dans cette spécification.

F25. Mode IA

L'option '-a [COLOR]', '--ai [=COLOR]' permet de lancer le programme en mode joueur artificiel avec la couleur COLOR ('X', 'O' et 'A' (All), 'O' par défaut).

4 Interface utilisateur

F26. Interface en ligne de commande

Le programme devra posséder une interface en ligne de commande qui permette une interaction avec l'utilisateur via le terminal. Cette interface textuelle sera lancée par défaut par le programme.

F27. Interface graphique

Le programme devra posséder une interface graphique qui permette les mêmes interactions que sur le CLI avec l'utilisateur via une fenêtre graphique. Cette interface graphique sera lancée si l'option '-g', '--gui' est donnée.

F28. Affichage du plateau

À chaque coup, le programme devra afficher le plateau ainsi que le temps pris par chaque joueur si la partie est en blitz.

En mode ASCII les pions de chaque joueur seront représentés par des caractères différents, 'X' pour les bleus et 'O' pour les rouges et les cases vides par un '.'.

F29. Notation des coups

Le programme devra permettre de déplacer les pions en utilisant la notation indiquées sur la figure suivante :

	a	b	c	d	e	f	g	h	i	j	k
1	.	.	.	.	.	.	.	0	.	.	1
2	.	.	.	.	.	.	.	.	.	.	2
3	.	.	.	.	.	.	.	.	.	.	3
4	.	.	.	.	.	.	.	.	.	.	4
5	.	.	.	.	.	.	.	.	.	.	5
6	X	.	.	.	.	.	.	.	.	.	6
7	.	.	.	.	.	.	.	.	.	.	7
8	.	.	.	.	.	.	.	.	.	.	8
9	.	.	.	.	.	.	.	.	.	.	9
10	.	.	.	.	.	.	.	.	.	.	10
11	.	.	.	.	.	.	.	.	.	.	11
	a	b	c	d	e	f	g	h	i	j	k

Le programme devra s'assurer que les règles sont suivies. Si un coup est illégal (comme placer un pion hors du plateau ou sur une case déjà occupée), un message d'erreur sera affiché et le programme demandera un autre coup à l'utilisateur.

F30. Représentation de l'historique

L'historique sera représenté comme une liste des tours du jeu. Par exemple, le premier tour sera affiché comme suit : '1. 0 h4 X e3' (le numéro du tour dans la partie, le joueur puis la case où est placé le nouveau pion). Les accolades '{}' signalent un commentaire qui sera ignoré par le parseur.

F31. Abandon

Le programme devra permettre à un joueur d'abandonner la partie en cours de jeu.

F32. Affichage des messages

Le programme devra afficher des messages d'information pendant la partie en fonction du mode dans lequel il est (default, verbose, debug).

F33. Quitter et sauvegarde une partie

Le programme devra permettre de quitter une partie en cours et de la sauvegarder dans un fichier.

F34. Sauvegarde de l'historique

Le programme devra pouvoir sauvegarder l'historique des coups joués dans un fichier à la demande de l'utilisateur.

F35. Naviguer dans l'historique

Il doit être possible de revenir en arrière ou d'aller en avant dans l'historique des coups joués si tous les joueurs humains sont d'accord.

F51. Configuration d'une nouvelle partie

L'item 'New Game' du menu 'File' permettra de mener l'utilisateur à une fenêtre de configuration de la partie qui sera déjà remplie avec les paramètres par défaut. Qu'ils soient modifiés ou non, l'utilisateur pourra lancer la partie en cliquant sur 'Start'.

8 Joueur artificiel

F52. Heuristique d'évaluation de position

Le programme devra comporter au moins une fonction capable de prendre un état du jeu et de lui donner un score en fonction de différents critères qui vous sembleront pertinents.

F53. Algorithme de recherche Minimax

Le choix du meilleur coup se fera via une exploration de l'arbre des coups possibles via l'algorithme de recherche Minimax. Cette option sera activée par défaut si le programme est lancé en mode joueur artificiel.

F54. $\alpha\beta$ -pruning

Le choix du meilleur coup se fera via une exploration de l'arbre des coups possibles via l'algorithme de l' $\alpha\beta$ -pruning. Cette option sera activée par défaut si le programme est lancé en mode joueur artificiel. Néanmoins, l'option '--ai-mode ab' sera tout de même disponible.

F55. Exploration peu profonde préliminaire

L'option '--ai-shallow' permettra d'activer une exploration préliminaire peu profonde de l'arbre des coups possibles puis d'ordonner les coups en fonction de leur intérêt, ceci pour tirer le maximum de profit de l' $\alpha\beta$ -pruning.

F56. Monte Carlo Tree Search

L'option '--ai-mode mcts' remplacera l'algorithme d'exploration par défaut par un algorithme de *Monte Carlo Tree Search* (MCTS).

F57. Profondeur de recherche

L'option '--ai-depth DEPTH' permettra de spécifier la profondeur de recherche de l'algorithme de recherche.

F58. Choix des heuristiques

Il doit être possible de gérer plusieurs heuristiques d'évaluation de position. L'option '--ai-heuristic HEURISTIC' permettra de spécifier l'heuristique choisie.

F59. Heuristique de fin de partie

Créer au moins une heuristique de fin de partie pour optimiser l'évaluation des positions.

F60. Temps de réflexion borné

L'option '--ai-time TIME' permettra de spécifier le temps de réflexion de l'algorithme de recherche. Si le temps est écoulé, le programme devra jouer le meilleur coup trouvé jusqu'à présent. Par défaut, le temps de réflexion sera de 5 secondes.

Références

- [1] Borderick Arneson, Ryan B. Hayward, and Philip Henderson. Monte-Carlo tree search in hex. *IEEE Transaction on Computational Intelligence and AI in Games*, 2(4) :251–257, December 2010.
- [2] Cameron Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.